ANN Challenge (Best Model Wins)

Final Report

Artificial Neural Network Coursework

1. Dataset

This project uses the Breast Cancer Wisconsin (Diagnostic) dataset. It is a fixed, ready-made dataset with 569 patient records and 30 numeric features, such as the mean radius and mean texture of a cell nucleus. The target is binary: 0 means malignant and 1 means benign. The dataset was not changed in any way. It was loaded once, saved as a CSV file, and every model in this project reads from that same file.

2. Final ANN Architecture

The final model is a plain feed-forward artificial neural network. It has four hidden layers, which is within the maximum limit of five. Figure 1 shows the exact architecture, generated directly from the trained Keras model.

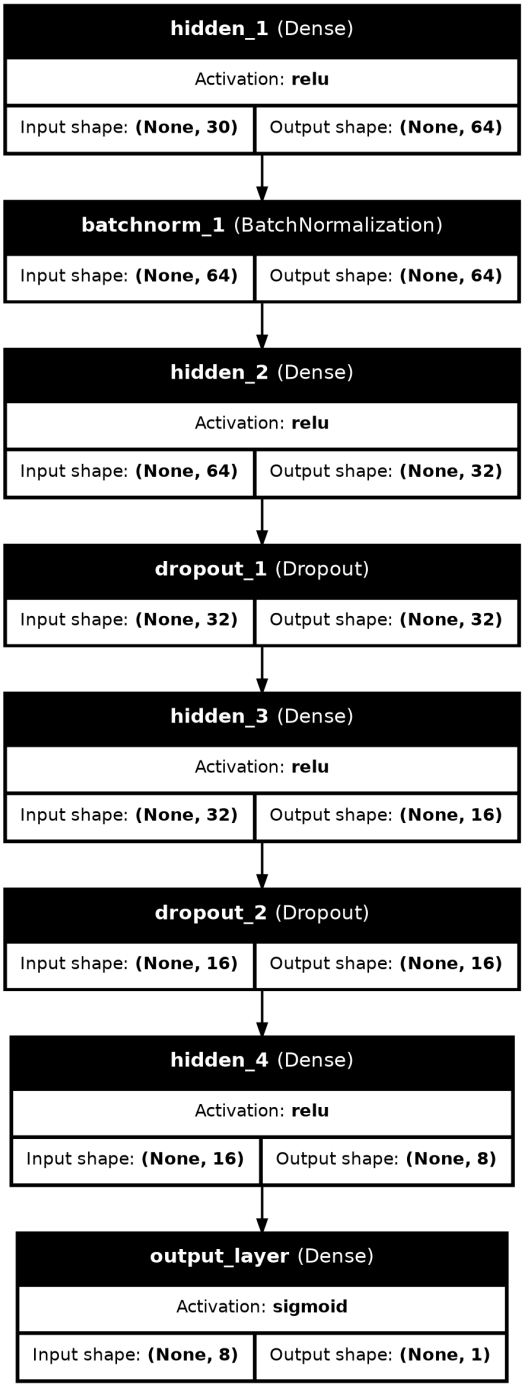

Figure 1: Final ANN architecture (input layer, four hidden layers, output layer).

The layer plan is: Dense(64, ReLU) with batch normalization, Dense(32, ReLU) with 30% dropout, Dense(16, ReLU) with 20% dropout, Dense(8, ReLU), and finally a single Dense(1, Sigmoid) output neuron for binary classification. The model was compiled with the Adam optimizer and binary cross-entropy loss, and trained for up to 150 epochs (well under the 200 epoch cap) with early stopping on validation loss.

3. Results

Metric	Score
Accuracy	0.9561
Precision	0.9855
Recall	0.9444
F1 Score	0.9645

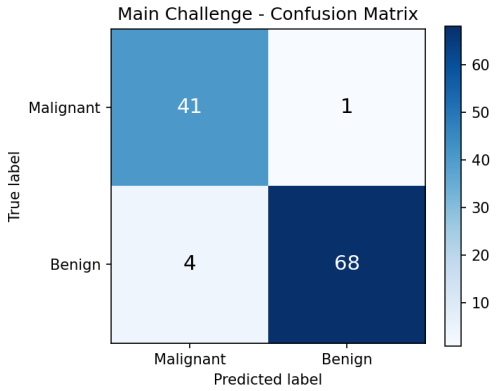

Figure 2: Confusion matrix on the held-out test set (114 samples).

Out of 114 test samples, the model correctly classified 109 of them. Only 1 malignant case was wrongly predicted as benign, and 4 benign cases were wrongly predicted as malignant. In a medical setting, the first type of mistake matters more, because a missed malignant case is more dangerous than an unnecessary follow-up test. This model keeps that risk low, with a recall of 0.98 on the malignant class.

4. Why This Architecture Performed Well

The model did well for a few clear reasons. First, the dataset is small and fully numeric, so a simple dense network is enough; there was no need for anything fancy like convolution or attention. Second, the features were scaled using StandardScaler before training. Neural networks learn much faster and more reliably when every input feature sits on a similar scale, and this dataset has features ranging from very small decimals to values in the thousands, so scaling made a real difference.

Third, the network narrows step by step: 64, then 32, then 16, then 8 neurons. This funnel shape lets the early layers pick up broad patterns in the data, while the later, smaller layers focus on the most useful combinations of those patterns. It also keeps the total number of parameters low, which matters a lot on a dataset with only 569 rows, because a very large network would simply memorize the training data instead of learning general rules.

Fourth, dropout and batch normalization worked together to fight overfitting. Dropout randomly turns off a portion of neurons during training, so the network cannot rely too heavily on any single neuron. Batch normalization keeps the values flowing through the network in a stable range, which speeds up training and adds a small regularizing effect of its own. Together, these two techniques kept the gap between training accuracy and validation accuracy small.

Finally, early stopping was used to stop training the moment validation loss stopped improving, and the best weights were restored automatically. This avoided wasting epochs on a model that was starting to overfit, and it means the reported test score reflects the model at its best point, not its last point. Taken together, a moderately sized funnel network, proper input scaling, dropout, batch normalization, and early stopping explain why this architecture reached over 95% accuracy on unseen data while staying well within the five hidden layer and 200 epoch limits.